学习总结：2026-10-05（RAG Day 1 —— 索引链路）

一、今日成果

- 从零搭起一条最小 RAG 索引链路：`demo.py`（3 句话验证语义检索）→ `ingest_course.py`（27,399 个课程 chunks 入库 Chroma）。
- 显式指定 embedding 模型 `all-MiniLM-L6-v2`（sentence-transformers），而非 Chroma 默认。
- Query "What is RAG?" 在课程语料上 top-3 全部相关，距离从 0.85 降到 0.36–0.41。**Task B 完成**。

二、遇到的困难 & 怎么解决的

1. **query 返回的 embeddings 是 None** 现象：`results["embeddings"]` 为 None，以为向量没生成。 解决：查清是 `include` 参数默认值不含 embeddings（向量存了，只是不返回）。`include=["embeddings", ...]` 可看。

2. **Batch size 27399 > max batch size 5461 报错** 现象：`collection.add()` 一次塞 27,399 条被拒。 解决：Chroma 单次 `add` 上限由 SQLite 单次 SQL 变量数推导而来；改成分批写入（每批 1000）。教训：ingest pipeline 里分批写是标准动作（backpressure）。

3. **Top-3 结果极烂**（CONTRIBUTING.md、行为准则、AGENTS.md 表格；距离全在 0.85 左右挤成一团） 诊断：glob 只扫了仓库顶层，喂进去的是脚手架文件而非课程正文；距离"又高又平"说明语料里根本没有答案。 解决：递归扫 `phases/**/*.md` + 排除脚手架文件。 教训：**Garbage in, garbage out —— 检索质量一半在语料清洗**。调试检索先看"召回的 chunk 是不是人话"，再看距离分布。

4. **每次重跑都要重新 embedding，又慢又浪费** 解决（自己提出来的）：`chromadb.Client()` 是纯内存、进程结束数据清空；换 `PersistentClient(path="./chroma_db")` 落盘，ingest 只跑一次。附带规则：**换 embedding 模型必须删库重建**（不同模型的向量空间不能混用）。

三、搞懂的核心概念

- **存查必须同模型**：写入和查询用同一个 embedding 模型，否则向量不在同一空间。
- **Chroma 内部结构**：SQLite 存元数据/文档原文（metadata segment）+ HNSW 存向量做相似度查找（vector segment，独立组件，不在 SQLite 里）。
- **HNSW**：多层图结构；build 时每个新向量找邻居连边（build 慢，O(n log n)）；查询时从顶层贪心游走（query 快，O(log n)）。近似最近邻（ANN），速度换精度。
- **通用模式**：把重计算前置到离线（预计算），在线只享受结果 —— ingest/query 分离的本质。

- **Chroma 两种形态**：library 模式（进程内函数调用，无网络）vs server 模式（`chroma run` + HttpClient）。Postgres 永远是 server。
- **SQLite vs pgvector**：不是谁更强，是场景不同 —— Chroma 选 SQLite 要的是零运维开箱即用；pgvector 给的是"生产已有 Postgres、不想再运维一套向量库"的人。

四、待解决问题：reranker 精排（下次继续）

背景：query="What is RAG?" 的 top-3 人工相关性裁判：

检索排名	内容	人工判断
Top 1 (0.3597)	RAGAS 是 RAG 评测框架.....	相关但答非所问（默认你已知 RAG 是什么）
Top 2 (0.3748)	RAG 系统四个部件：chunker, retriever, reranker, generator.....	最对题（直接定义）
Top 3 (0.4083)	如何评估 RAG，三个指标.....	相关但跑偏

人工排序应为 Top 2 > Top 1 > Top 3，但 bi-encoder 给出 Top 1 > Top 2 > Top 3。

核心问题：reranker 如何提供最精确的答案（把 Top 2 抬到第一）？

理论要点：

- bi-encoder：query 和文档各自压成 384 维向量再比距离；文档向量在没见过 query 时已算好，"RAGAS"天然躺在"RAG"附近 —— 比的是"意思像不像"（有损压缩的代价）。
- cross-encoder：把 "query [SEP] 文档" 整句喂进 transformer，attention 在两者词之间直接交互；在 MS MARCO 上学到"答没答到题" —— 判的是"定义句 vs 同名工具介绍"。
- 前者可离线预计算（快），后者必须在线逐对过模型（慢）→ retrieve-then-rerank 两阶段设计。

待运行代码（加在 `ingest_course.py` 末尾，`n_results` 改为 10）：

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")  # 首次运行会下载模型
docs = results["documents"][0]
pairs = [(QUERY, d) for d in docs]
scores = reranker.predict(pairs)

ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
print("\n=== reranker 精排后 Top-3 ===")
for i, (d, s) in enumerate(ranked[:3]):
    print(f"--- 第{i+1}名 (score={s:.4f}) ---")
```

```
    print(d[:200])
    print()
```

待验证：cross-encoder 的实际打分是否把 Top 2 抬到第一；注意分数与距离方向相反（分数越大越好，距离越小越好）。

五、English Summary

What I built: A minimal RAG indexing pipeline from scratch — `demo.py` (3-sentence sanity check) → `ingest_course.py` (27,399 course chunks embedded with all-MiniLM-L6-v2, stored in Chroma). Top-3 for "What is RAG?" all relevant; distances dropped from ~0.85 to 0.36–0.41.

Difficulties & fixes:

1. `embeddings=None` in query results → just the default `include` param; vectors are stored, not returned. Fixed with `include=["embeddings", ...]`.
2. `Batch size 27399 > max 5461` → Chroma's per-`add` limit (derived from SQLite's variable cap); fixed with batched writes (1,000/batch).
3. Terrible top-3 (scaffolding files; flat distances ~0.85) → glob had only scanned top level; corpus had no answer. Fixed with recursive `phases/` glob + exclusion list. Lesson: garbage in, garbage out.
4. Re-embedding on every run → switched from ephemeral `chromadb.Client()` to `PersistentClient`; ingest once. Rule: changing the embedding model requires rebuilding the index.

Concepts: same model for index & query; Chroma = SQLite (metadata) + HNSW (separate vector index); HNSW builds neighbors at index time (slow build, fast query; approximate NN); offline precompute vs online serving; Chroma library vs server mode; SQLite vs pgvector is a scenario tradeoff, not a ranking.

Open question: reranker — bi-encoder ranked a RAGAS-eval chunk above the definitional answer (0.3597 vs 0.3748); cross-encoder reranking (retrieve-then-rerank) should fix the ordering. Code ready, run pending.

六、面试官追问 + 参考答案（基于今日真实经历）

Q1. "Walk me through your RAG indexing pipeline."

中文思路：一句话总览 → 分四步讲（每步带一个决策和原因）→ 结尾点出 ingest/query 分离。

English answer: "Sure. I built a course Q&A assistant over a local 'AI engineering from scratch' course repo. The pipeline has four stages. First, file collection — I recursively globbed the course's phases directory for markdown and excluded scaffolding files like CONTRIBUTING and the code

of conduct, because my first run ingested those and retrieval quality was garbage. Second, chunking — split on blank lines and dropped fragments under 50 characters. Third, embeddings — I explicitly pinned all-MiniLM-L6-v2 via sentence-transformers rather than Chroma's default, so the model used at index time is guaranteed identical to query time. Fourth, storage — batched writes of 1,000, because Chroma rejects single adds over about 5,400 records, into a PersistentClient so the index survives restarts. The whole ingest runs offline once; at query time only the query gets embedded."

Q2. "Your top-3 distances were flat at ~0.85. Walk me through your debugging."

中文思路：先亮判断框架（先查数据再查模型）→ 两个证据（chunk 内容不对 + 距离又高又平）→ 修复与验证 → 金句收尾。

English answer: "My rule is: when retrieval is bad, suspect the data before the model. Two signals told me it was a corpus problem, not a model problem. First, the returned chunks themselves — CONTRIBUTING.md, a code of conduct — weren't course content at all; my glob had only scanned the repo root. Second, the distance distribution: all three results were flat around 0.85. If the corpus contained a good answer, you'd expect at least one result clearly closer than the rest. Flat and high means nothing in the index is near the query — the answer simply isn't there. So I fixed file collection to recursively scan the course phases directory and exclude scaffolding files. After re-ingesting, distances dropped to 0.36–0.41 and all three results were on-topic. The lesson: half of retrieval quality is corpus curation — garbage in, garbage out."

Q3. "Why move from ephemeral to persistent Chroma client? What breaks if you switch embedding models after launch?"

中文思路：前半答浪费（27k 重算）→ 后半答向量空间（silent failure 是关键词）→ 生产 checklist。

English answer: "I started with the default in-memory client, which meant every script rerun re-embedded all 27,000 chunks — minutes wasted each time. Since document embeddings only need to be computed once, I switched to PersistentClient so the index lives on disk and ingest runs exactly once; at query time only the query gets embedded. As for switching models after launch — that silently breaks everything. Embeddings from different models live in different vector spaces, so distances between an old document vector and a new query vector are meaningless. You must wipe and rebuild the index. In production I'd make that an explicit checklist item in the model-upgrade runbook, because the failure mode is silent degradation, not an error."

Q4. "Chroma rejected your 27k-record write. Root cause and fix?"

中文思路：root cause 定位到 SQLite 变量上限 → fix 分批 → 升华到 ingest 常识。

English answer: "The root cause was Chroma's per-add batch limit — around 5,400 records — which is derived from SQLite's cap on bound variables per statement, since Chroma stores

metadata in SQLite. My corpus had 27,000 chunks, so a single add was rejected. The fix was straightforward: loop over the chunks in batches of 1,000. More generally, it taught me that batched writes with backpressure handling are table stakes for any ingest pipeline — you never assume the store accepts unbounded writes."

Q5. "Why does Chroma use SQLite for metadata but a separate HNSW index for vectors? When would you pick pgvector instead?"

中文思路：先讲分工（各干各的）→ 再讲选型（答"场景不同"不答"谁更强"）→ 两句话收尾。

English answer: "They serve different jobs. SQLite stores the small, structured side — ids, raw documents, metadata — where a relational store is plenty. Vector search is handled by a separate HNSW index, which organizes vectors into a multi-layer graph so queries run in logarithmic time instead of scanning everything. As for pgvector versus Chroma — it's not about which is better, it's about the team's stage. Chroma's embedded mode is zero-ops: pip install and you're running, ideal for prototyping. pgvector makes sense when production already runs Postgres and you'd rather not operate a second database — vectors co-located with business data, one system to maintain, at the cost of running Postgres yourself."

Q6. "HNSW is approximate. When would that hurt you, and how would you know?"

中文思路：什么时候 hurt（距离接近时误差翻转排序）→ 用今天的真实数字举例（0.3597 vs 0.3748）→ 怎么发现（距离分布 + recall 评测）→ 缓解手段（调参 / reranker，呼应待解决问题）。

English answer: "It hurts when the true nearest neighbors are close together — approximation error can flip their order. I saw a hint of this myself: my top two results sat at distances 0.3597 and 0.3748, a gap of 0.015. At that margin you can't tell whether the ranking reflects the true ordering or graph-traversal noise. How would I know in production? Two signals: a flat distance distribution across the top-k, where nothing clearly separates from the rest; and offline recall evaluation against brute-force exact search on a sample. Mitigations are widening the search scope at query time, or — what I did next — adding a cross-encoder reranker over the top candidates, which re-scores each query-document pair exactly."